06066300 แนวคิดระบบฐานข้อมูล · คณะเทคโนโลยีสารสนเทศ สจล.

Online Car Rental System

ระบบเช่ารถยนต์ออนไลน์



สิทธิพันธุ์ ฮงเจริญ
68070182

จิราธิวัชร ทะวิลา
68070228

ภรัมพรหม เหลืองภุมรัตน์
68070229

พูนพิพัฒน์ ช่างเกวียน
68070329

อาจารย์ผู้สอน ผศ.ดร.พัฒนพงษ์ ฉันทมิตรโอภาส

ที่มาและความสำคัญของปัญหา



จดบันทึกกระดาษ

ข้อมูลกระจายหลายแหล่ง ไม่เชื่อมโยงกัน ค้นหาช้า



ข้อมูลซ้ำซ้อน

ลูกค้า รถ ราคา สถานะ บันทึกซ้ำและคลาดเคลื่อน



จองซ้อน

รถคันเดียวถูกจองในช่วงเวลาทับซ้อน (Double Booking)



ต้องโทรถาม

ลูกค้าตรวจสอบสถานะรถเองไม่ได้ เพิ่มภาระพนักงาน



ทางออก ฐานข้อมูลกลางที่รวมลูกค้า รถ การจอง การเช่า และการชำระเงินไว้ในระบบเดียว ตรวจสถานะได้ทันที และป้องกันการจองซ้อนอัตโนมัติ

วัตถุประสงค์และขอบเขต

วัตถุประสงค์

1 ศึกษาและวิเคราะห์กระบวนการเช่ารถ แล้วนำหลักการฐานข้อมูลมาออกแบบ

2 ออกแบบระบบให้จัดเก็บ สืบค้น ตรวจสอบ และปรับปรุงข้อมูลได้อย่างเป็นระบบ

3 ลดความซ้ำซ้อนและความคลาดเคลื่อนในการจัดการรถ การจอง การรับและคืนรถ

ขอบเขตของระบบ



ลูกค้า (Customer)
สมัคร ค้นหารถ จอง ชำระเงิน ติดตามสถานะ



ผู้ดูแลระบบ (Admin)
จัดการรถ ประเภทรถ ราคา การจอง การรับ-คืนรถ

6 ตารางหลัก

CUSTOMER	CAR_TYPE	CAR
BOOKING	RENTAL	PAYMENT

ความต้องการของผู้ใช้ (User Requirements)

Functional Requirements

ระบบลูกค้า CRUD ข้อมูลลูกค้า และตรวจสอบเลขใบขับขี่ (LicenseNo)

ระบบรถยนต์ จัดหมวดหมู่ Sedan / SUV / Van / Eco Car และอัปเดตสถานะรถ

ระบบการเช่า กำหนดวันรับ-คืนรถ คำนวณ TotalAmount อัตโนมัติ

ระบบชำระเงิน แบ่งจ่ายได้หลายครั้ง: มัดจำ ค่าเช่า ค่าปรับ

Non-functional

Security

ข้อมูลใบขับขี่และข้อมูลส่วนบุคคลต้องเข้ารหัสและเก็บอย่างปลอดภัย

Data Integrity

ห้ามจองรถคันเดียวกันในช่วงเวลาทับซ้อน และวันคืนต้องหลังวันรับ

การเลือกใช้ซอฟต์แวร์และสถาปัตยกรรม

PostgreSQL 16
บน Supabase (Managed Cloud)

- ข้อมูลมีความสัมพันธ์ชัดเจน เหมาะกับ Relational Database
- รองรับ CHECK, FOREIGN KEY และ EXCLUDE ใช้กันการจองซ้อนได้ในตัว
- รองรับ Transaction (ACID), CTE และ Window Function
- Supabase ให้ใช้ฟรี ออนไลน์ 24 ชั่วโมง มี SQL Editor ในเว็บ

Frontend
React / Flutter บน Vercel

↓

Backend API
RESTful API ด้วย FastAPI / Node.js

↓

Database
PostgreSQL 16 (Supabase)

เครื่องมือ Mermaid / draw.io (ERD) | pgAdmin / Supabase SQL

การทำงานของระบบ 5 ขั้นตอน



ขั้นที่ 1

ลงทะเบียน

บันทึกลูกค้าและตรวจใบขับขี่

CUSTOMER



ขั้นที่ 2

ค้นหารถ

แสดงเฉพาะรถที่ Status = Available

CAR · CAR_TYPE



ขั้นที่ 3

จองและเช่า

TotalAmount = จำนวนวัน × DailyRate รถเปลี่ยนเป็น Rented

BOOKING · RENTAL



ขั้นที่ 4

ชำระเงิน

บันทึกการจ่าย การจองเป็น Confirmed

PAYMENT



ขั้นที่ 5

คืนรถ

Admin ตรวจสภาพ RENTAL = Completed รถกลับเป็น Available

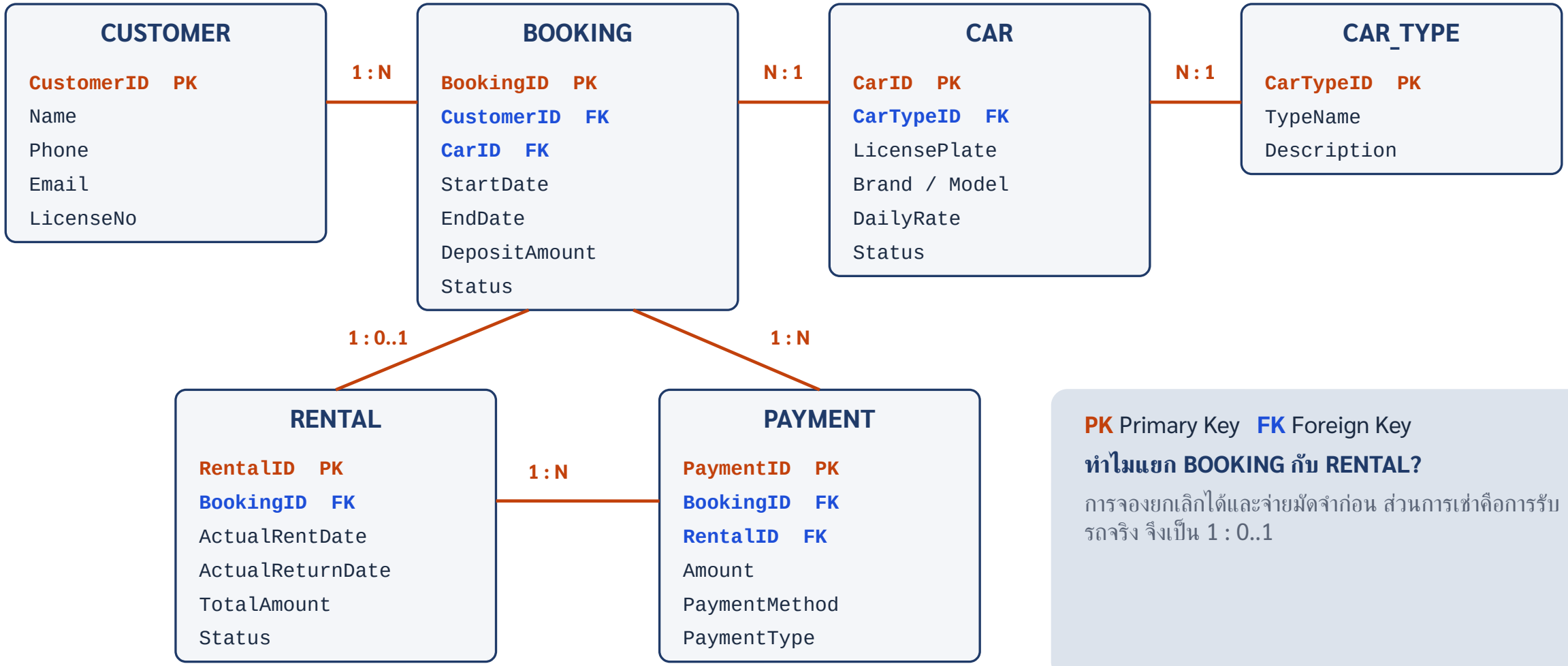
RENTAL · CAR

สถานะรถ: Available → Rented → Available สถานะการจอง: Pending → Confirmed → Completed

กฎทางธุรกิจ (Business Rules) → Constraint

กฎ	ความหมาย	บังคับด้วย
BR1	ลูกค้า 1 คนเช่าได้หลายรายการ แต่ 1 รายการเป็นของลูกค้า 1 คน	`FK customer_id ใน BOOKING (1:N)`
BR2	รถ 1 คันห้ามถูกจองซ้อนในช่วงเวลาเดียวกัน	`EXCLUDE constraint no_overlap`
BR3	รถแต่ละคันอยู่ใน 1 ประเภท ประเภทหนึ่งมีรถได้หลายคัน	`FK car_type_id NOT NULL`
BR4	การเช่าต้องระบุรถ วันที่เช่า และวันที่คืน	`NOT NULL car_id, start_date, end_date`
BR5	การเช่า 1 รายการชำระได้หลายครั้ง (มัดจำ + ส่วนที่เหลือ)	`PAYMENT → BOOKING (1:N)`
BR6	เช่าได้เฉพาะรถที่สถานะ Available	กรองสถานะใน `query` ค้นหารถว่าง
BR7	วันที่คืนต้องไม่อยู่ก่อนวันที่เช่า	`CHECK (end_date > start_date)`

Entity Relationship Diagram (ERD)



PK Primary Key **FK** Foreign Key

ทำไมแยก BOOKING กับ RENTAL?

การจองยกเลิกได้และจ่ายมัดจำก่อน ส่วนการเช่าคือการรับรถจริง จึงเป็น 1 : 0..1

Normalization: 1NF → 2NF → 3NF

1NF

ตาราง BOOKING มีข้อมูลลูกค้าปนอยู่ (Name, Phone, Email, LicenseNo) และ CAR มี TypeName

ค่าทุกช่องเป็นค่าเดี่ยว แต่ยังซ้ำซ้อน

2NF

แยก CUSTOMER ออกจาก BOOKING

ตัด Partial Dependency: ข้อมูลลูกค้าขึ้นกับ CustomerID ไม่ใช่ BookingID

3NF

แยก CAR_TYPE ออกจาก CAR

ตัด Transitive Dependency: CarID → CarTypeID → TypeName

ผลลัพธ์: 6 ตาราง ไม่มี Insertion / Update / Deletion Anomaly

ก่อน (1NF)

```
BOOKING  BookingID, CarID, StartDate, EndDate, Status, Name, Phone, Email, LicenseNo
```

```
CAR  CarID, CarTypeID, Brand, Model, DailyRate, TypeName
```

หลัง (3NF)

BOOKING	CUSTOMER
CAR	CAR_TYPE
RENTAL	PAYMENT

Data Dictionary

ตาราง	PK	FK
CUSTOMER	CustomerID	–
CAR_TYPE	CarTypeID	–
CAR	CarID	CarTypeID
BOOKING	BookingID	CustomerID, CarID
RENTAL	RentalID	BookingID
PAYMENT	PaymentID	BookingID, RentalID

ชนิดข้อมูลที่ใช้

- INT / SERIAL
- VARCHAR(n)
- DECIMAL(10,2)
- TIMESTAMP

ตัวอย่างรายละเอียด: ตาราง BOOKING

Field	Type	Key	คำอธิบาย
BookingID	INT	PK	รหัสการจอง
CustomerID	INT	FK	ลูกค้าที่จอง
CarID	INT	FK	รถที่จอง
BookingDate	TIMESTAMP		วันเวลาที่จอง
StartDate	TIMESTAMP		วันเริ่มเช่า
EndDate	TIMESTAMP		วันสิ้นสุดการเช่า
DepositAmount	DECIMAL(10,2)		เงินมัดจำ
Status	VARCHAR(20)		สถานะการจอง

สร้างฐานข้อมูลด้วย SQL DDL

```sql
CREATE TABLE booking (
  booking_id  SERIAL PRIMARY KEY,
  customer_id INT NOT NULL REFERENCES customer,
  car_id      INT NOT NULL REFERENCES car,
  start_date  TIMESTAMP NOT NULL,
  end_date    TIMESTAMP NOT NULL,
  deposit_amount DECIMAL(10,2) DEFAULT 0,
  status VARCHAR(20) DEFAULT 'Pending'
    CHECK (status IN ('Pending','Confirmed',
                      'Cancelled','Completed')),
  CHECK (end_date > start_date),        -- BR7
  EXCLUDE USING gist (car_id WITH =,    -- BR2
    tsrange(start_date, end_date) WITH &&)
    WHERE (status <> 'Cancelled')
);
```



PK / FK
เชื่อม customer และ car



CHECK
จำกัดค่าสถานะ และวันคืน > วันรับ (BR7)



EXCLUDE
รถคันเดิม + ช่วงเวลาซ้อน = ปฏิเสธทันที (BR2)



INDEX
car(status), booking(car_id) ค้นหาเร็วขึ้น

6 ตาราง **30** constraints

ตัวอย่างข้อมูล (Sample Data)

4	8	6	8	6	12
ประเภทรถ	คัน	ลูกค้า	การจอง	การเช่า	การชำระเงิน

รถ	ประเภท	บาท/วัน	Status
Toyota Camry	Sedan	1,800	**Available**
Honda Civic	Sedan	1,500	**Rented**
Toyota Fortuner	SUV	2,500	**Available**
Toyota Commuter	Van	3,000	**Available**
Hyundai H-1	Van	2,800	**Maintenance**
Nissan Almera	Eco Car	900	**Available**

ข้อมูลครอบคลุมทุกกรณีทดสอบ

- สถานะรถทั้ง 3 แบบ
- สถานะการจองทั้ง 4 แบบ รวม Cancelled
- ชำระหลายงวด: มัดจำ + ค่าเช่า + ค่าปรับ
- การเช่าที่ยังไม่คืน เพื่อทดสอบยอดค้างชำระ

การทดสอบระดับง่าย (Easy) · 6 คำสั่ง

```sql
-- E1 available cars by price
SELECT brand, model, daily_rate
FROM car WHERE status = 'Available'
ORDER BY daily_rate;
```

brand	model	daily_rate
Nissan	Almera	900.00
Toyota	Yaris Ativ	950.00
Toyota	Camry	1800.00
Toyota	Fortuner	2500.00
Toyota	Commuter	3000.00

ผลลัพธ์ 5 แถว

E1 **SELECT + WHERE + ORDER BY**
รถที่ว่าง เรียงตามราคา → 5 คัน

E2 **SELECT WHERE**
ค้นหาลูกค้าจากเลขใบขับขี่

E3 **เงื่อนไขช่วงราคา**
รถค่าเช่า ≤ 2,000 บาท → 4 คัน

E4 **INSERT**
เพิ่มลูกค้าใหม่

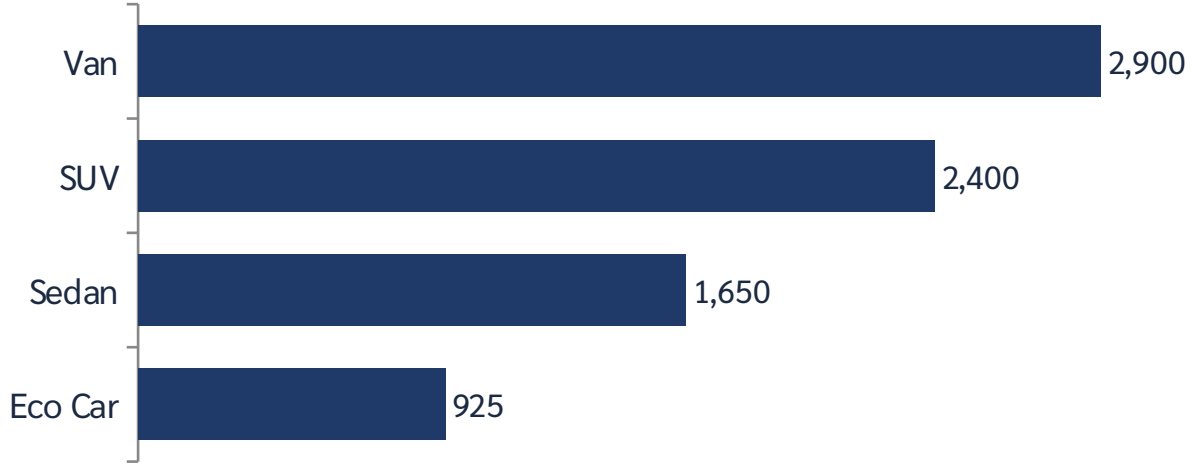
E5 **UPDATE**
รถซ่อมเสร็จ → Available

E6 **DELETE**
ลบลูกค้าที่ไม่เคยจอง

การทดสอบระดับปานกลาง (Medium) · 5 คำสั่ง

```sql
-- M2 cars and avg rate per type
SELECT t.type_name, COUNT(c.car_id) AS cars,
       ROUND(AVG(c.daily_rate), 2) AS avg_rate
FROM car_type t LEFT JOIN car c
  ON c.car_type_id = t.car_type_id
GROUP BY t.type_name ORDER BY avg_rate DESC;
```

ผล M2: ค่าเช่าเฉลี่ยต่อวันตามประเภท



M1 — **JOIN 3 ตาราง**
การจองพร้อมชื่อลูกค้าและรถ (8 แถว)

M2 — **LEFT JOIN + GROUP BY + AVG**
Van 2,900 · SUV 2,400 · Sedan 1,650

M3 — **SUM + GROUP BY**
Credit Card 15,900 · Cash 9,500 · QR 3,800

M4 — **GROUP BY + HAVING**
ลูกค้าที่จองมากกว่า 1 ครั้ง (2 คน)

M5 — **JOIN 4 ตาราง**
รถที่กำลังถูกเช่าและวันกำหนดคืน

การทดสอบระดับซับซ้อน (Complex) · 7 คำสั่ง

```sql
-- C1 outstanding balance per rental (CTE)
WITH paid AS (
  SELECT booking_id, SUM(amount) AS paid
  FROM payment WHERE status = 'Paid'
  GROUP BY booking_id)
SELECT r.rental_id,
  r.total_amount - COALESCE(p.paid, 0) AS outstanding
FROM rental r LEFT JOIN paid p USING (booking_id)
WHERE r.total_amount - COALESCE(p.paid, 0) > 0;
```

rental_id	customer	total	outstanding
5	พิมพ์ชนก	10,500	**9,500**
6	วีรภัทร	6,900	**5,400**

ผล C1 (ก่อนทำรายการคืนรถ C5): พบยอดค้างชำระ 2 รายการ

C1 — **CTE + LEFT JOIN**
ยอดค้างชำระ 9,500 และ 5,400

C2 — **Window: RANK() OVER**
อันดับรายได้ #1 Honda Civic 10,500

C3 — **NOT EXISTS + tsrange &&**
หารถว่างตามช่วงวันที่ต้องการ

C4 — **Subquery ใน HAVING**
ลูกค้าที่จ่ายสูงกว่าค่าเฉลี่ย

C5 — **BEGIN … COMMIT**
คืนรถครบทั้ง 4 ตารางใน 1 Transaction

C6–C7 — **Constraint test**
ทดสอบ BR2 และ BR7 (สไลด์ถัดไป)

ทดสอบความถูกต้องของข้อมูล (Constraint Test)

BR2 · ห้ามจองซ้อน

จองรถ Camry 21–24 ต.ค. ซ้อนกับการจองเดิม 20–23 ต.ค.

```
INSERT INTO booking (customer_id,
  car_id, start_date, end_date)
VALUES (3, 1, '2026-10-21 10:00',
               '2026-10-24 10:00');
```

ERROR: conflicting key value violates exclusion constraint "no_overlap"

BR7 · วันคืนต้องหลังวันรับ

วันคืน 1 พ.ย. อยู่ก่อนวันรับ 5 พ.ย.

```
INSERT INTO booking (customer_id,
  car_id, start_date, end_date)
VALUES (3, 3, '2026-11-05',
               '2026-11-01');
```

ERROR: new row violates check constraint "chk_booking_dates"

ฐานข้อมูลปฏิเสธข้อมูลผิดได้เอง ไม่ต้องพึ่งโค้ดฝั่งแอปพลิเคชัน

สรุปและประเมินผล

6	7	30	18
ตาราง ถึง 3NF	Business Rules	Constraints	คำสั่งทดสอบผ่าน

ผลที่ได้

- ฐานข้อมูลครบทุกความต้องการของผู้ใช้
- ป้องกันการจองซ้อนและวันที่ผิดในระดับฐานข้อมูล
- ทดสอบครบทั้งง่าย กลาง ซับซ้อน

ข้อจำกัด

- ยังไม่มีหน้าเว็บสำหรับผู้ใช้จริง
- ยังไม่ได้เข้ารหัสข้อมูลใบขับขี่
- BR6 ยังตรวจใน query ไม่ใช่ constraint

แนวทางพัฒนาต่อ

- พัฒนา Web App เชื่อม Supabase
- Trigger อัปเดตสถานะรถอัตโนมัติ
- รีวิวรถ โปรโมชั่น และ Dashboard รายได้



ขอบคุณครับ

Q & A

สิทธิพันธุ์ ฮงเจริญ 68070182 · จิราธิวัชร ทะวิลา 68070228
ภรัมพรหม เหลืองภุมรัตน์ 68070229 · พูนพิพัฒน์ ช่างเกวียน 68070329